AI WEATHERWISE

Intelligent Weather Forecasts and AI-Powered Weather Insights

---

PHASE 4 – PROJECT PLANNING

Project Planning and Development Schedule

Project Type
AI-powered weather information and recommendation REST API / backend

Technology Stack
Node.js · Express.js · MongoDB Atlas · Mongoose · JWT · bcrypt · Google Gemini AI · REST APIs · Thunder Client / Postman

Document Date
September 28, 2026

Status
Project Planning Phase

BSc Computer Science Project

TABLE OF CONTENTS

1. Project Plan

1.1 Project Objective

The objective of AI WeatherWise is to design and build a secure REST API backend that delivers real-time weather information together with AI-generated weather summaries and recommendations. The system converts raw weather metrics such as temperature, humidity, wind speed and condition into natural-language guidance that users can act on directly, for example when choosing clothing or planning outdoor activities.

To achieve this, the backend is planned to:

- Allow users to register and log in, with passwords hashed using bcrypt before they are stored.
- Use JWT authentication so that user-specific resources are available only to authenticated users.
- Retrieve real-time weather information for a requested city by integrating an external weather service (OpenWeatherMap), with a resilient fallback response when the external service key is not configured.
- Allow logged-in users to create, view, update and delete their own favorite locations.
- Integrate Google Gemini AI to provide AI-powered weather summaries and weather-related recommendations or insights based on current weather details.
- Store user and favorite-location information in MongoDB Atlas using Mongoose schemas.
- Return standard JSON responses and handle validation problems and errors through a centralized approach.

The project is a backend API only. Its features are exercised and verified through API testing tools such as Thunder Client or Postman.

1.2 Project Scope

In Scope

- User registration and login, with JWT authentication and password hashing (bcrypt / bcryptjs).

- Protected API access: authentication middleware verifies a JWT before private routes are used, while weather lookup remains available as a public feature.
- Weather information retrieval for a city through an external weather API (OpenWeatherMap), including resilient fallback behavior where supported.
- Favorite location management: create, view, update and delete favorite locations, scoped to the authenticated user.
- Google Gemini AI integration for AI weather summaries and AI weather recommendations/insights.
- Data storage in MongoDB Atlas using Mongoose (User and Location collections).
- Input validation, centralized error handling, request sanitization and standard JSON responses.
- API testing using Thunder Client / Postman.

Out of Scope

- A graphical web or mobile frontend. The project is a backend API only, exercised through API testing tools.
- Weather hardware or sensors; weather data is obtained from an external weather API.
- Training or building custom machine-learning models. AI features rely on the existing Google Gemini AI service.
- Native mobile application development.

1.3 Development Approach

AI WeatherWise follows an incremental, module-by-module development approach. Work moves from planning and requirement analysis to design, then to project setup, and then builds one module at a time. After each module is implemented, its endpoints are tested with Thunder Client / Postman before the next module begins, so that problems are found early and are isolated to a single module.

The planned sequence of development stages is:

1. Project planning
2. Requirement analysis
3. Project design

4. Backend project setup
5. Database configuration
6. Authentication implementation
7. Weather API integration
8. Favorite location module
9. Gemini AI integration
10. Validation and error handling
11. API testing
12. Final documentation
13. Demonstration

The layered MVC-style structure documented for the project (routes, middleware, controllers, services and models) is followed throughout. Code is planned to be kept under version control with Git / GitHub so that progress remains traceable.

1.4 API-Based Implementation Approach

AI WeatherWise is a REST API backend. Every feature is exposed through HTTP endpoints that accept and return JSON, and there is no template-based user interface. Endpoints are organized into logical groups and are planned to be built in the order below. Functional descriptions are used here instead of fixed endpoint URLs, because the exact routes belong to the design and implementation phases.

1. **Authentication APIs –** user registration, login and protected access. Passwords are hashed with bcrypt, and a signed JWT is issued on login and verified by authentication middleware on private routes.
2. **Weather APIs –** retrieval of current weather information (temperature, humidity, wind speed and condition) for a city. A weather service communicates with the external weather API, and a fallback response keeps the endpoint usable when the external key is not configured. This feature is public and does not require an account.
3. **Location APIs –** favorite location management (create, view, update and delete), restricted to the logged-in user's own records through a reference to the owning user.

4. **AI APIs –** AI weather summary and AI weather recommendation/insight. The controller passes weather details such as city, temperature, humidity and condition to a Gemini service, which calls Google Gemini AI and returns the generated text as JSON.
5. **Common API Handling –** input validation, centralized error handling, request sanitization and a consistent JSON response format shared by all endpoints, so that each new API can be tested in the same way.

1.5 Technology Selection

The following table lists the technologies selected for AI WeatherWise and the reason each one suits this project.

Technology	Reason for Selection
Node.js	Server-side JavaScript runtime suited to building an API backend that handles many requests, with a large package ecosystem.
Express.js	Minimal framework that provides routing, middleware and JSON request handling for the weather, location, authentication and AI routes.
MongoDB Atlas	Cloud-hosted document database used to persist WeatherWise user and favorite-location data without a local database installation.
Mongoose	Provides schemas, validation rules and the reference between a saved location and its owning user on top of MongoDB.
JWT	Stateless token-based authentication that lets protected routes verify users without server-side sessions.
bcrypt / bcryptjs	Hashes passwords one-way so that plain-text passwords are never stored in the database.
Google Gemini AI	Generates natural-language weather summaries and condition-aware recommendations from weather details.
External Weather API (OpenWeatherMap)	Supplies live current-weather metrics such as temperature, humidity, wind speed and condition.
Thunder Client / Postman	Sends API requests and checks responses without needing a frontend.
Git / GitHub	Version control and remote backup, keeping module-by-module progress traceable.

1.6 Development Journey: Setup to Demonstration

The project moves through the following stages, from planning to final demonstration:

Planning → Requirement Analysis → Project Design → Backend Setup → Dependencies and Environment Configuration → MongoDB Atlas Connection → Authentication → Weather API Integration → Favorite Location Module → Gemini AI Integration → Validation and Error Handling → API Testing → End-to-End Testing → Documentation → Final Demonstration.

The project begins with planning, requirement analysis and design. Development then starts with backend setup: the folder structure, dependencies, environment variables (such as the server port, database connection string, JWT secret and Gemini API key) and the Git repository. The backend is planned to run on port 5001. The MongoDB Atlas connection is then configured so that data can be stored.

Authentication is built first because favorite locations and other user-specific operations depend on a logged-in user. Weather API integration comes next, because weather data must be available before AI insights can be generated from it. The favorite location module follows and is connected to authenticated users through the user reference. The Gemini AI integration is built after the weather module, since the AI service uses weather details such as city, temperature, humidity and condition as its input.

Validation and error handling are then applied across all modules. Once every module exists, the API is tested endpoint by endpoint and then end to end, the documentation is finalized, and the project is presented in a final demonstration.

1.7 Major Activities and Expected Deliverables

Activity	Purpose	Expected Deliverable
Project Planning	Define project direction, scope, schedule and resources	Project plan
Requirement Analysis	Identify functional and non-functional requirements	Requirement document
Project Design	Design the architecture, database and user flow	Project design document
Backend Setup	Configure the Node.js / Express project, dependencies and environment variables	Working backend structure
Database Setup	Configure MongoDB Atlas and Mongoose schemas	Working database connection
Authentication	Implement registration, login, password hashing and JWT protection	Authentication module
Weather Integration	Connect the external weather service, with fallback behavior	Weather functionality
Favorite Locations	Manage saved cities for each authenticated user	Location functionality
Gemini AI Integration	Generate AI weather summaries and recommendations	AI functionality
Validation and Error Handling	Validate input, sanitize requests and standardize errors	Consistent validated API behavior
Testing	Verify API behavior with Thunder Client / Postman	Tested API operations
Documentation	Prepare the project documentation	Final documentation
Demonstration	Present the working project	Final project demonstration

2. Task and Timeline Plan

The project is planned over **12 weeks**. Durations are estimates for a student project, and no calendar dates are assumed. A few tasks overlap slightly where a later task can begin as soon as part of its dependency is finished.

2.1 Project Timeline Table

Task ID	Task Description	Dependency	Estimated Duration	Deliverable
T1	Project Planning	None	1 week	Project plan
T2	Requirement Analysis	T1	1 week	Requirement document
T3	Project Design	T2	1 week	Project design document
T4	Backend Project Setup	T3	0.5 week	Working backend structure
T5	Database Configuration	T4	0.5 week	MongoDB Atlas connection and Mongoose setup
T6	Authentication Module	T5	1 week	Registration, login and JWT protection
T7	Weather API Integration	T6 / T4	1 week	Weather retrieval with fallback behavior
T8	Favorite Location Module	T6 / T5	1 week	Favorite location management
T9	Gemini AI Integration	T7	1 week	AI weather summaries and recommendations
T10	Validation and Error Handling	T6 / T7 / T8 / T9	1 week	Validated inputs and centralized error handling
T11	API Testing	T6 / T7 / T8 / T9 / T10	1 week	Tested API operations
T12	End-to-End Testing	T11	0.5 week	End-to-end test results
T13	Documentation	T12	1 week	Final documentation
T14	Final Demonstration	T13	0.5 week	Project demonstration

2.2 Gantt Chart

ID	Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
T1	Project Planning												
T2	Requirement Analysis												
T3	Project Design												
T4	Backend Setup												
T5	Database Config												
T6	Authentication												
T7	Weather API												
T8	Favorite Locations												
T9	Gemini AI												
T10	Validation & Errors												
T11	API Testing												
T12	End-to-End Testing												
T13	Documentation												
T14	Final Demonstration												

Figure 1: AI WeatherWise – Project Timeline / Gantt Chart (Week 1 to Week 12)

Reading the chart: tasks run top to bottom in development order. Planning and design tasks (T1–T3) come first, followed by backend setup and database configuration (T4–T5), which share Week 4. Authentication (T6) comes before the favorite location module because saved locations belong to a logged-in user. Weather API integration (T7) precedes Gemini AI integration (T9) because the AI features work from weather details. Validation and error handling (T10) is applied once the main modules exist, and API testing (T11) and end-to-end testing (T12) share Week 10. Documentation (T13) follows in Week 11, and the project ends with the final demonstration (T14) in Week 12.

3. Resource Planning

A. Hardware Resources

AI WeatherWise is a backend API, so development requires a machine that can run the Node.js server, a code editor, a web browser and an API testing tool at the same time. The figures below are practical project-planning estimates and are not requirements stated by the WeatherWise documentation.

Resource	Minimum	Recommended	Purpose
Computer / Laptop	Dual-core 64-bit processor	Quad-core (Intel i5 / Ryzen 5 or equivalent)	Runs the Node.js server, code editor, browser and API testing tool together.
RAM	4 GB	8 GB or more	Smooth multitasking between editor, browser, testing tool and MongoDB-related tools.
Storage	10 GB free	256 GB SSD with 20 GB or more free	Operating system, tools, project files and node_modules.
Internet Connection	Stable broadband	Stable broadband	Required for MongoDB Atlas, the Google Gemini API, the external weather API, npm packages and GitHub.

B. Software Resources

Software	Purpose
Windows / Linux / macOS	Operating system for development; any of the three is suitable for running the AI WeatherWise backend.
Node.js (LTS version)	Runtime used to execute the WeatherWise backend server, planned to run on port 5001.
npm	Installs and manages project dependencies such as Express, Mongoose, jsonwebtoken, bcryptjs, dotenv, CORS and the Google Gemini SDK.
Visual Studio Code (or equivalent editor)	Writing, running and debugging the WeatherWise code.
MongoDB Atlas	Cloud database hosting for the User and Location collections.
Thunder Client / Postman	Sending API requests and verifying responses for authentication, weather, favorite location and AI endpoints.
Git / GitHub	Version control and remote backup of the project source code.
Web browser	Accessing MongoDB Atlas, API provider dashboards for keys, documentation and GitHub.

C. Development Technologies

Technology	Role in AI WeatherWise
JavaScript	Programming language used across the whole backend.
Node.js	Server-side runtime that executes the API.
Express.js	Web framework for routes, middleware and controllers.
Mongoose	Object modeling for MongoDB: schemas, validation and the reference from a favorite location to its user.
MongoDB Atlas	Stores user accounts and favorite locations.
JWT	Token creation and verification for protected endpoints.
bcrypt / bcryptjs	Password hashing and comparison during registration and login.
Google Gemini AI (API)	Source of AI weather summaries and recommendations; requires a Gemini API key stored in environment variables.
External Weather API / OpenWeatherMap	Source of live current-weather data (temperature, humidity, wind speed and condition); its API key is also kept in environment variables.

Technology	Role in AI WeatherWise
REST API	Architectural style used to expose all features as HTTP endpoints returning JSON.
Thunder Client / Postman	Testing tools used to verify API requests and responses.